```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;
using System.IO;
using BambooEnterprise.Models;
using BambooEnterprise.Data;
using BambooEnterprise.Testing;

namespace BambooEnterprise
{
    class Program
    {
        static List<BambooSpecies> SpeciesDb = new();
        static List<InventoryBatch> InventoryDb = new();
        static List<PerformanceLog> GrowthDb = new();
        static List<OperationalExpense> ExpenseDb = new();
        private static readonly HttpClient _httpClient = new();

        static void Main(string[] args)
        {
            try
            {
                // Verify database existence at startup
                string invPath = Path.Combine("Data", "inventory.json");
                if (File.Exists(invPath))
                {
                    Console.WriteLine("=== SYSTEM STARTUP ===");
                    Console.Write("Database files detected. Load existing data? (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        LoadAllData();
                        Console.WriteLine("Data loaded successfully.");
                    }
                    else
                    {
                        SeedSampleData();
                        Console.WriteLine("System initialized with Hardcoded Sample Data.");
                    }
                }
                else
                {
                    SeedSampleData();
                    Console.WriteLine("No database found. System initialized with Hardcoded Sample Data.");
```

```
                }

                Console.WriteLine("Press any key to enter Main Menu...");
                Console.ReadKey();
                MainMenu();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Startup Error: {ex.Message}");
                Console.ReadKey();
            }
        }

        static void MainMenu()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("=== BAMBOO INFORMATION MANAGEMENT SYSTEM ===");
                ShowActiveAlerts();
                Console.WriteLine("\n1. Manage Biological Profiles (DNA)");
                Console.WriteLine("2. Track Inventory & Acquisition");
                Console.WriteLine("3. Log Growth & Harvests");
                Console.WriteLine("4. Operational Expenses (OpEx)");
                Console.WriteLine("5. Financial Reports (P&L)");
                Console.WriteLine("6. Reset/Re-Seed Database");
                Console.WriteLine("Q. Quit Application");
                Console.Write("\nSelect: ");

                string choice = Console.ReadLine()?.ToUpper();
                if (choice == "Q") return;

                try { HandleChoice(choice); }
                catch (Exception ex) when (ex.Message == "MainMenu") { continue; }
                catch (Exception ex)
                {
                    Console.WriteLine($"\nApplication Error: {ex.Message}");
                    Console.WriteLine("Press any key to return to menu...");
                    Console.ReadKey();
                }
            }
        }

        static void HandleChoice(string choice)
```

```
        {
            switch (choice)
            {
                case "1": ManageSpeciesFlow(); break;
                case "2": ManageInventoryFlow(); break;
                case "3": ManageGrowthFlow(); break;
                case "4": ManageExpenseFlow(); break;
                case "5":
                    PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);
                    PersistenceManager.ExportToExcel(InventoryDb, "Inventory_Report");
                    Console.WriteLine("\nReports generated in Reports/ folder (PDF and Excel).");
                    Console.ReadKey(); break;
                case "6": SeedSampleData(); break;
                default:
                    // Informed user of invalid input
                    Console.WriteLine("\n!!! Invalid selection. Please enter 1-6 or Q.");
                    Console.ReadKey();
                    break;
            }
        }

        static void ShowActiveAlerts()
        {
            Console.ForegroundColor = ConsoleColor.Yellow;
            int month = DateTime.Now.Month;
            int year = DateTime.Now.Year;

            if (month >= 5 && month <= 7)
                Console.WriteLine("!!! ALERT: Peak EDIBLE SHOOT harvest season (May-July) !!!");
            if (month >= 10 || month <= 2)
                Console.WriteLine("!!! ALERT: Peak TIMBER harvest season (Autumn-Winter) !!!");

            foreach (var b in InventoryDb.Where(batch => batch.IsPeakCulmAge(year)))
                Console.WriteLine($"> Batch {b.BatchID} ({b.PlantUID}) reached PEAK CULM MATURITY (4+ yrs).");

            Console.ResetColor();
        }

        #region Inventory Logic
        static void ManageInventoryFlow()
        {
```

```
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 2. TRACK INVENTORY ---");
                Console.WriteLine("1. View All | 2. Summary View | 3. Add | 4. Update | 5. Delete | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();

                if (cmd == "B")
                {
                    // Confirmation flow for saving
                    Console.Write("\nSave inventory changes to database? (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                        Console.WriteLine("Changes saved.");
                        System.Threading.Thread.Sleep(1000);
                    }
                    return;
                }

                if (cmd == "1") ViewInventory();
                if (cmd == "2") ViewInventorySummary();
                if (cmd == "3") AddInventoryFlow().GetAwaiter().GetResult();
                if (cmd == "4") UpdateInventoryFlow();
                if (cmd == "5") DeleteInventoryFlow();
            }
        }

        static void ViewInventory()
        {
            Console.WriteLine("\nBATCH ID | PLANT UID | QUANTITY | TOTAL COST");
            foreach (var b in InventoryDb)
                Console.WriteLine($"{b.BatchID} | {b.PlantUID} | {b.Quantity} units | {b.TotalCostSek:N2} SEK");
            Console.ReadKey();
        }

        static void ViewInventorySummary()
        {
            Console.Clear();
            Console.WriteLine("Group by: 1. Nursery | 2. Date | 3. Species Name");
            string mode = Console.ReadLine();
            string exportContent = $"INVENTORY SUMMARY -
```

```
                {DateTime.Now:yyyy-MM-dd HH:mm}\n" + new string('=', 50) + "\n";

            if (mode == "1")
            {
                foreach (var g in InventoryDb.GroupBy(b => b.Nursery).OrderBy(x => x.Key))
                {
                    double totalInvest = g.Sum(b => b.TotalCostSek);
                    string header = $"\nNURSERY: {g.Key.ToUpper()} | Total Investment: {totalInvest:N2} SEK";
                    Console.WriteLine(header);
                    exportContent += header + "\n";

                    foreach (var b in g)
                    {
                        string line = $" [{b.BatchID}] {b.PlantUID} - {b.Quantity} units ({b.TotalCostSek:N2} SEK)";
                        Console.WriteLine(line);
                        exportContent += line + "\n";
                    }
                }
            }
            else if (mode == "2")
            {
                foreach (var g in InventoryDb.GroupBy(b => b.PurchaseDate.Date).OrderBy(x => x.Key))
                {
                    string header = $"\nDATE: {g.Key:yyyy-MM-dd}";
                    Console.WriteLine(header);
                    exportContent += header + "\n";
                    foreach (var b in g) Console.WriteLine ($" [{b.BatchID}] {b.Nursery} - {b.Quantity} units");
                }
            }
            else if (mode == "3")
            {
                var joined = InventoryDb.Join(SpeciesDb, b => b.PlantUID, s => s.PlantUID, (b, s) => new { b, s });
                foreach (var g in joined.GroupBy(x => $"{x.s.Genus} {x.s.Species}").OrderBy(x => x.Key))
                {
                    string header = $"\nSPECIES: {g.Key}";
                    Console.WriteLine(header);
                    exportContent += header + "\n";
                    foreach (var item in g) Console.WriteLine ($" [{item.b.BatchID}] {item.b.Nursery} - {item.b.Quantity} units");
```

```
                }
            }

            Console.WriteLine("\nExport this summary report? (Y/N)");
            if (Console.ReadLine()?.ToUpper() == "Y")
            {
                PersistenceManager.ExportSummary(exportContent, "Inventory_Summary");
                Console.WriteLine("Report saved to Reports/Summary folder.");
            }
            Console.WriteLine("\nPress any key to return...");
            Console.ReadKey();
        }

        static async Task AddInventoryFlow()
        {
            DateTime date = ValidateDate(RequestInput("Date (YYYY-MM-DD)", true));
            string nursery = RequestInput("Nursery Name", true);
            string nurseryCode = (nursery.Length >= 3 ? nursery.Substring(0, 3) : nursery.PadRight(3, 'X')).ToUpper();

            string prefix = $"{date:yyyyMMdd}-{nurseryCode}";
            int sequence = InventoryDb.Count(x => x.BatchID.StartsWith(prefix)) + 1;
            string batchId = $"{prefix}-{sequence:D3}";

            Console.WriteLine($"Suggested Batch ID: {batchId}");
            string finalId = RequestInput("Press Enter to accept or type custom ID", false);
            if (string.IsNullOrEmpty(finalId)) finalId = batchId;

            string uid = RequestInput("Plant UID (DNA Link)", true);
            if (!SpeciesDb.Any(s => s.PlantUID == uid)) throw new Exception("UID not found in biological profiles.");

            string currency = RequestInput("Currency (e.g. EUR)", true).ToUpper();
            double rate = 1.0;
            if (currency != "SEK")
            {
                rate = await FetchExchangeRate(currency, date);
                Console.WriteLine($"Fetched Rate: 1 {currency} = {rate:N4} SEK");
                string manualRate = RequestInput("Accept rate? (Enter new value or press Enter)", false);
                if (!string.IsNullOrEmpty(manualRate)) rate = ValidatePositiveDouble(manualRate);
```

```
            }

            InventoryDb.Add(new InventoryBatch
            {
                BatchID = finalId,
                PlantUID = uid,
                PurchaseDate = date,
                Nursery = nursery,
                Quantity = ValidatePositiveInt(RequestInput("Quantity", true)),
                UnitCost = ValidatePositiveDouble(RequestInput("Unit Cost", true)),
                ExchangeRateToSek = rate,
                Curr = Enum.TryParse(currency, out Currency c) ? c : Currency.SEK
            });
            // Note: Saving now happens on "Back" in ManageInventoryFlow
        }

        static void UpdateInventoryFlow()
        {
            string bid = RequestInput("Enter Batch ID to update", true);
            var b = InventoryDb.FirstOrDefault(x => x.BatchID == bid);
            if (b == null) throw new Exception("Batch not found.");

            string newQty = RequestInput($"New Qty (Current: {b.Quantity})", false);
            if (!string.IsNullOrEmpty(newQty)) b.Quantity = ValidatePositiveInt(newQty);
        }

        static void DeleteInventoryFlow()
        {
            string bid = RequestInput("Batch ID to delete", true);
            var b = InventoryDb.FirstOrDefault(x => x.BatchID == bid);
            if (b != null) InventoryDb.Remove(b);
        }
        #endregion

        #region Species Logic
        static void ManageSpeciesFlow()
        {
            Console.Clear();
            Console.WriteLine("--- BIOLOGICAL PROFILES (DNA) ---");
            Console.WriteLine("S. Search Filter | V. View All | B. Back");
            string cmd = Console.ReadLine()?.ToUpper();
            if (cmd == "B") return;

            IEnumerable<BambooSpecies> displayList = SpeciesDb;
```

```
            if (cmd == "S")
            {
                Console.Write("Enter search term (Name or Hardiness): ");
                string filter = Console.ReadLine()?.ToLower();
                displayList = SpeciesDb.Where(s => ($"{s.Genus} {s.Species} {s.Cultivar}").ToLower().Contains(filter)
                    || s.HardinessLow.ToString().Contains(filter));
            }

            int count = 0;
            foreach (var s in displayList)
            {
                if (count % 10 == 0)
                {
                    Console.WriteLine("\n{0,-12} {1,-42} {2,-7} {3,-7} {4}", "UID", "Scientific Name \"Cultivar\"", "Hardy", "Max Ht", "Research Notes");
                    Console.WriteLine(new string('-', 95));
                }
                string name = $"{s.Genus} {s.Species} \"{s.Cultivar}\"".Trim();
                Console.WriteLine("{0,-12} {1,-42} {2,-7} {3,-7} {4}", s.PlantUID, name, s.HardinessLow + "C", s.HeightHigh + "m", s.SverigeSpecifikt);
                count++;
            }
            Console.WriteLine("\nPress any key to return...");
            Console.ReadKey();
        }
        #endregion

        #region Growth and Expenses
        static void ManageGrowthFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 3. GROWTH ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;

                if (cmd == "1")
                {
                    foreach (var l in GrowthDb) Console.WriteLine($"{l.Date:yyyy-MM} | Batch {l.BatchID} | Ht: {l.MaxHeightCurrent}m");
                    Console.ReadKey();
                }
```

```
                if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID", true);
                    GrowthDb.Add(new PerformanceLog
                    {
                        LogID = Guid.NewGuid().ToString().Substring(0, 8),
                        BatchID = bid,
                        Date = DateTime.Now,
                        NewShootCount = ValidatePositiveInt(RequestInput("Shoots", true)),
                        MaxHeightCurrent = ValidatePositiveDouble(RequestInput("Ht", true)),
                        CulmRevenueSek = ValidatePositiveDouble(RequestInput("Culm Rev", false) ?? "0"),
                        ShootRevenueSek = ValidatePositiveDouble(RequestInput("Shoot Rev", false) ?? "0")
                    });
                    PersistenceManager.SaveToJson(GrowthDb, "logs.json");
                }
            }
        }

        static void ManageExpenseFlow()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 4. EXPENSES ---");
                Console.WriteLine("1. View | 2. Add | B. Back");
                string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;

                if (cmd == "1")
                {
                    foreach (var e in ExpenseDb) Console.WriteLine($"{e.Date:yyyy-MM} | {e.BatchID} | {e.MaintenanceCost} SEK");
                    Console.ReadKey();
                }
                if (cmd == "2")
                {
                    string bid = RequestInput("Batch ID (or 'Global')", true);
                    ExpenseDb.Add(new OperationalExpense
                    {
                        ExpenseID = Guid.NewGuid().ToString().Substring(0, 8),
                        BatchID = bid,
                        Date = DateTime.Now,
```

```
                        MaintenanceCost = ValidatePositiveDouble(RequestInput("Maintenance", false) ?? "0"),
                        LaborCost = ValidatePositiveDouble(RequestInput("Labor", false) ?? "0"),
                        EquipmentCost = ValidatePositiveDouble(RequestInput("Equipment", false) ?? "0"),
                        TransportCost = ValidatePositiveDouble(RequestInput("Transport", false) ?? "0")
                    });
                    PersistenceManager.SaveToJson(ExpenseDb, "expenses.json");
                }
            }
        }
        #endregion

        #region Helpers and Data
        static void SeedSampleData()
        {
            SpeciesDb = SeedDataGenerator.GetResearchedSpecies();
            InventoryDb = SeedDataGenerator.GetSampleInventoryBatches();
            PersistenceManager.SaveToJson(SpeciesDb, "species.json");
            PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
        }

        static void LoadAllData()
        {
            SpeciesDb = PersistenceManager.LoadFromJson<BambooSpecies>("species.json");
            InventoryDb = PersistenceManager.LoadFromJson<InventoryBatch>("inventory.json");
            GrowthDb = PersistenceManager.LoadFromJson<PerformanceLog>("logs.json");
            ExpenseDb = PersistenceManager.LoadFromJson<OperationalExpense>("expenses.json");
        }

        private static async Task<double> FetchExchangeRate(string currency, DateTime date)
        {
            try
            {
                string url = $"https://frankfurter.app{date:yyyy-MM-dd}?from={currency}&to=SEK";
                var response = await _httpClient.GetStringAsync(url);
                using var doc = JsonDocument.Parse(response);
                return doc.RootElement.GetProperty("rates").GetProperty("SEK").GetDouble();
```

```
            }
            catch { return 1.0; }
        }

        static string RequestInput(string p, bool r)
        {
            Console.Write($"{p} (B back, M main): ");
            string i = Console.ReadLine();
            if (i?.ToUpper() == "B") throw new Exception("GoBack");
            if (i?.ToUpper() == "M") throw new Exception("MainMenu");
            if (string.IsNullOrEmpty(i) && r) return RequestInput(p, r);
            return i;
        }

        static double ValidatePositiveDouble(string i) => double.TryParse (i, out double r) && r >= 0 ? r : throw new Exception("Invalid number.");
        static int ValidatePositiveInt(string i) => int.TryParse(i, out int r) && r >= 0 ? r : throw new Exception("Invalid integer.");
        static DateTime ValidateDate(string i) => DateTime.TryParse(i, out DateTime r) ? r : throw new Exception("Invalid date.");
        #endregion
    }
}

```

```
using System;
using System.Collections.Generic;
using BambooEnterprise.Models;

namespace BambooEnterprise.Testing
{
    public static class SeedDataGenerator
    {
        // Hardcoded Sample Inventory Data Moved to the top for easy management
        public static readonly List<InventoryBatch> SampleInventory = new List<InventoryBatch>
        {
            new InventoryBatch {
                BatchID = "20220611-KIM-001",
                PlantUID = "PHY-VIV-AUR",
                PurchaseDate = new DateTime(2022, 06, 11),
                Nursery = "Kimmei",
                PotVolume = 10,
                Quantity = 2,
                UnitCost = 45.00,
                Curr = Currency.EUR,
                ExchangeRateToSek = 11.20
            },
            new InventoryBatch {
                BatchID = "20220611-KIM-002",
                PlantUID = "PHY-PAR-NOM",
                PurchaseDate = new DateTime(2022, 06, 11),
                Nursery = "Kimmei",
                PotVolume = 5,
                Quantity = 1,
                UnitCost = 35.00,
                Curr = Currency.EUR,
                ExchangeRateToSek = 11.20
            }
        };

        public static List<InventoryBatch> GetSampleInventoryBatches() => SampleInventory;

        public static List<BambooSpecies> GetResearchedSpecies()
        {
            var list = new List<BambooSpecies>();
            // Runners
            list.Add(CreateMono("PHY-VIV-HUA", "Phyllostachys", "vivax", "Huanwenzhu", 12, 10, -21, 6, "Green culm, yellow sulcus. Snöbrott risk."));
            list.Add(CreateMono("PHY-VIV-MCC", "Phyllostachys", "vivax", "McClure", 14, 12, -21, 6, "Classic green timber. Large
```

```
                leaves."));
            list.Add(CreateMono("PHY-VIV-AUR", "Phyllostachys", "vivax", "Aureocaulis", 12, 11, -21, 6, "Yellow culm, green stripes. High aesthetics."));
            list.Add(CreateMono("PHY-PAR-NOM", "Phyllostachys", "parvifolia", "", 10, 9, -25, 5, "Extremely hardy timber. Edible shoots."));
            list.Add(CreateMono("PHY-ATR-INC", "Phyllostachys", "atrovaginata", "Incense", 8, 7, -23, 5, "Air canals for wet soil. Scented."));
            list.Add(CreateMono("PHY-AS-NOM", "Phyllostachys", "aureosulcata", "", 7, 4, -25, 5, "Green culm, yellow sulcus. Zig-zag bases."));
            list.Add(CreateMono("PHY-AS-AUR", "Phyllostachys", "aureosulcata", "Aureocaulis", 7, 4, -25, 5, "Solid yellow. Very hardy."));
            list.Add(CreateMono("PHY-AS-SPE", "Phyllostachys", "aureosulcata", "Spectabilis", 7, 4, -25, 5, "Yellow culm, green sulcus. Red shoots."));
            list.Add(CreateMono("PHY-AS-ARG", "Phyllostachys", "aureosulcata", "Argus", 7, 4, -25, 5, "Green stripes on yellow."));
            list.Add(CreateMono("PHY-NIG-NOM", "Phyllostachys", "nigra", "", 5, 3, -18, 5, "Jet black culms. Needs shelter."));
            list.Add(CreateMono("PHY-NIG-HEN", "Phyllostachys", "nigra", "Henonis", 10, 8, -23, 6, "Giant grey timber. Very hardy."));
            list.Add(CreateMono("PHY-NIG-PUN", "Phyllostachys", "nigra", "Punctata", 7, 4, -20, 6, "Speckled black culms."));
            list.Add(CreateMono("PHY-BIS-NOM", "Phyllostachys", "bissetii", "", 7, 4, -26, 4, "Best Swedish windbreak."));
            list.Add(CreateMono("PHY-DEC-NOM", "Phyllostachys", "decora", "", 6, 4, -22, 5, "Drought tolerant runner."));
            list.Add(CreateMono("PHY-NUD-NOM", "Phyllostachys", "nuda", "", 6, 4, -26, 4, "Early purple/black nodes."));
            list.Add(CreateMono("SAS-KUR-NOM", "Sasa", "kurilensis", "", 2, 1.5, -30, 6, "Northernmost species. Large leaves."));
            list.Add(CreateMono("BAS-QIN-NOM", "Bashania", "qingchengshanensis", "", 4, 2, -20, 4, "Strong, stiff culms."));
            list.Add(CreateMono("SEM-FAS-NOM", "Semiarundinaria", "fastuosa", "", 8, 4, -20, 6, "Upright columnar habit."));
            // Clumpers
            list.Add(CreateSym("FAR-NIT-OBE", "Fargesia", "nitida x murieliae", "Obelisk", 4, -25, 5, "Tallest upright columnar hedge."));
            list.Add(CreateSym("FAR-MUR-RZB", "Fargesia", "murieliae", "Red Zebra", 3, -23, 5, "Dark red/green contrast stems."));
            list.Add(CreateSym("FAR-NIT-SCH", "Fargesia", "nitida x murieliae", "Schensbossen", 4, -26, 5, "Exceptional
```

```
                hardiness."));
            list.Add(CreateSym("FAR-SPP-RUF", "Fargesia", "sp.", "Rufa", 2.5, -23, 4, "Non-curling leaves. Early shoots."));
            list.Add(CreateSym("FAR-ROB-CAM", "Fargesia", "robusta", "Campbell", 5, -20, 4, "Earliest shoots. White pattern."));
            list.Add(CreateSym("FAR-NIT-JUR", "Fargesia", "nitida", "Jürgen", 3.5, -26, 5, "Upright, dark culms."));
            list.Add(CreateSym("FAR-JIU-ONE", "Fargesia", "sp. Jiuzhaigou", "1", 3, -25, 5, "Red Panda. Cherry stems in sun."));
            list.Add(CreateSym("FAR-JIU-DPU", "Fargesia", "sp. Jiuzhaigou", "Deep Purple", 3, -25, 5, "Deepest purple culm color."));
            list.Add(CreateSym("FAR-DEM-GER", "Fargesia", "demissa", "Gerry", 3, -25, 5, "Almost black culms."));
            list.Add(CreateSym("FAR-DEN-XI2", "Fargesia", "denudata", "Xian 2", 4, -22, 5, "Cloud-like elegant habit."));
            list.Add(CreateSym("FAR-SPP-KR5", "Fargesia", "sp.", "KR 5287", 4, -20, 6, "Rare collector blueish culms."));
            return list;
        }



        private static Monopodial CreateMono(string uid, string g, string s, string c, double h, double d, double hard, int shoot, string note)
            => new Monopodial { PlantUID = uid, Genus = g, Species = s, Cultivar = c, Habit = GrowthHabit.Monopodial, HeightHigh = h, DiameterHigh = d, HardinessLow = hard, TypicalShootingMonth = shoot, SverigeSpecifikt = note, ShootEdibilityRank = 4, HeightLow = h * 0.7, DiameterLow = d * 0.6 };

        private static Sympodial CreateSym(string uid, string g, string s, string c, double h, double hard, int shoot, string note)
            => new Sympodial { PlantUID = uid, Genus = g, Species = s, Cultivar = c, Habit = GrowthHabit.Sympodial, HeightHigh = h, HardinessLow = hard, TypicalShootingMonth = shoot, SverigeSpecifikt = note, ClumpSpacing = 1.0, HeightLow = h * 0.7 };
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace BambooEnterprise.Models
{
    public enum GrowthHabit { Monopodial, Sympodial }
    public enum Currency { SEK, EUR, USD, GBP }

    [JsonDerivedType(typeof(Monopodial), typeDiscriminator: "monopodial")]
    [JsonDerivedType(typeof(Sympodial), typeDiscriminator: "sympodial")]
    public abstract class BambooSpecies
    {
        public string PlantUID { get; set; }
        public string Genus { get; set; }
        public string Species { get; set; }
        public string Cultivar { get; set; }
        public GrowthHabit Habit { get; set; }
        public double HeightLow { get; set; }
        public double HeightHigh { get; set; }
        public double DiameterLow { get; set; }
        public double DiameterHigh { get; set; }
        public double HardinessLow { get; set; }
        public int TypicalShootingMonth { get; set; }
        public string SverigeSpecifikt { get; set; }
        public int ShootEdibilityRank { get; set; }
        public int TimberQualityRank { get; set; }
    }

    public class Monopodial : BambooSpecies { public double RhizomeDepth { get; set; } }
    public class Sympodial : BambooSpecies { public double ClumpSpacing { get; set; } }

    public class InventoryBatch
    {
        public string BatchID { get; set; }
        public string PlantUID { get; set; }
        public DateTime PurchaseDate { get; set; }
        public string Nursery { get; set; }
        public double PotVolume { get; set; }
        public int Quantity { get; set; }
        public double UnitCost { get; set; }
        public Currency Curr { get; set; }
        public double ExchangeRateToSek { get; set; }
        public double TotalCostSek => (UnitCost * Quantity) * ExchangeRateToSek;
        public bool IsPeakCulmAge(int currentYear) => (currentYear - PurchaseDate.Year) >= 4;
```

```
    }

    public class PerformanceLog
    {
        public string LogID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public int NewShootCount { get; set; }
        public double MaxHeightCurrent { get; set; }
        public double CulmRevenueSek { get; set; }
        public double ShootRevenueSek { get; set; }
        public double EdibleKg { get; set; }
    }

    public class OperationalExpense
    {
        public string ExpenseID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public double MaintenanceCost { get; set; }
        public double EquipmentCost { get; set; }
        public double LaborCost { get; set; }
        public double TransportCost { get; set; }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;
using BambooEnterprise.Models;
using ClosedXML.Excel;
using iText.IO.Font.Constants;
using iText.Kernel.Font;
using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
using iText.Layout.Properties;

namespace BambooEnterprise.Data
{
    public static class PersistenceManager
    {
        private const string DataDir = "Data";
        private const string PdfDir = "Reports/PDF";
        private const string ExcelDir = "Reports/Excel";
        private const string SummaryDir = "Reports/Summary";

        static PersistenceManager()
        {
            Directory.CreateDirectory(DataDir);
            Directory.CreateDirectory(PdfDir);
            Directory.CreateDirectory(ExcelDir);
            Directory.CreateDirectory(SummaryDir);
        }

        public static void SaveToJson<T>(List<T> data, string fileName)
        {
            string path = Path.Combine(DataDir, fileName);
            File.WriteAllText(path, JsonSerializer.Serialize(data, new JsonSerializerOptions { WriteIndented = true }));
        }

        public static List<T> LoadFromJson<T>(string fileName)
        {
            string path = Path.Combine(DataDir, fileName);
            if (!File.Exists(path)) return new List<T>();
            try { return JsonSerializer.Deserialize<List<T>>(File.ReadAllText(path)) ?? new List<T>(); }
            catch { return new List<T>(); }
        }

        public static void ExportSummary(string content, string name)
        {
```

```
            string path = Path.Combine(SummaryDir, $"{name}_{DateTime.Now:yyyyMMdd}.txt");
            File.WriteAllText(path, content);
        }

        public static void ExportToExcel<T>(List<T> data, string name)
        {
            string path = Path.Combine(ExcelDir, $"{name}.xlsx");
            using (var wb = new XLWorkbook())
            {
                var ws = wb.Worksheets.Add("Report");
                ws.Cell(1, 1).InsertTable(data);
                wb.SaveAs(path);
            }
        }

        public static void GeneratePLReport(List<InventoryBatch> batches, List<PerformanceLog> logs, List<OperationalExpense> expenses)
        {
            string path = Path.Combine(PdfDir, "Bamboo_ProfitLoss_Report.pdf");
            using (var writer = new PdfWriter(path))
            using (var pdf = new PdfDocument(writer))
            using (var doc = new Document(pdf))
            {
                PdfFont bold = PdfFontFactory.CreateFont(StandardFonts.HELVETICA_BOLD);
                doc.Add(new Paragraph("Bamboo Enterprise - P&L Statement").SetFont(bold).SetFontSize(18));
                Table table = new Table(4).SetWidth(UnitValue.CreatePercentValue(100));
                table.AddHeaderCell("Batch ID"); table.AddHeaderCell("Revenue (SEK)");
                table.AddHeaderCell("Costs (SEK)"); table.AddHeaderCell("Net Profit");

                foreach (var b in batches)
                {
                    double rev = logs.Where(l => l.BatchID == b.BatchID).Sum(l => l.CulmRevenueSek + l.ShootRevenueSek);
                    double exp = expenses.Where(e => e.BatchID == b.BatchID || e.BatchID == "Global").Sum(e => e.MaintenanceCost + e.LaborCost + e.EquipmentCost + e.TransportCost);
                    double totalCost = exp + b.TotalCostSek;
                    table.AddCell(b.BatchID); table.AddCell(rev.ToString("N2"));
                    table.AddCell(totalCost.ToString("N2")); table.AddCell
```

```
                        ((rev - totalCost).ToString("N2"));
                }
                doc.Add(table);
            }
        }
    }
}

```